

Ta3awen

Security & Data Audit Report

Flutter + Firebase Mutual-Aid Marketplace (MVP)

Project: ta3awen-mvp-5a5ea | Date: 2026-10-05 | Auditor: automated audit (opencode agent)

Scope: security, data integrity and Firebase configuration only. No UI redesign, no new product features, no architecture rewrite. All fixes preserve current application behaviour; no billing or Firebase Storage migration introduced.

Verdict: MVP-ready from a data-security standpoint after the fixes in this report. Open items are listed in *Remaining Risks* (rate limiting, App Check, backups, seed cleanup).

1. Architecture Overview

The app is a single Flutter (Android APK) client talking directly to Firebase. There is no custom backend: Firebase Authentication issues identity tokens and Cloud Firestore Security Rules are the server-side authorization layer.

Components

- **Client:** Flutter 3.29 / Dart 3.7, obfuscated release builds, 3 locales (ar default / fr / en), light+dark theme, Android arm64-v8a published on GitHub Releases.
- **Auth:** Firebase Authentication, email/password only (no anonymous sign-in). A profile document `users/{uid}` is created at registration; `admins/{uid}` membership documents exist only for moderators.
- **Database:** Firestore collections `listings` (with subcollection `images`), `users`, `reports`, `admins`; 12 composite indexes for the faceted feed.
- **Images:** JPEG -> base64 inside Firestore documents (Cloud Storage requires billing on new projects). Thumbnail (480px, q55) lives in the parent listing as `image0`; full images ($\leq 1000\text{px}$, q70, ≤ 5 files) live in `listings/{id}/images/{index}`.
- **Search:** prefix-keyword index (`keywords` array) built client-side, queried with server-side `arrayContainsAny` (≤ 30 tokens) + status equality filters, cursor pagination.

Authorization model

Every write path is enforced by Firestore Rules (client UI gating is convenience only):

```
signedIn()      -> request.auth != null
isAdmin()       -> exists(admins/{request.auth.uid})      # console-managed only
canWrite()      -> signedIn && profile.disabled != true    # disabled accounts frozen
listings.create -> owner == auth.uid, status == available-literal (Arabic), server timestamps,
                  content validation (title/desc/phone/keywords/image0 caps)
listings.update -> admin OR (owner && ownerId unchanged && createdAt unchanged
                  && status in {available, given})
listings.delete -> admin OR owner
admins/*        -> write always false (Firebase console / Admin SDK only)
reports.read/up -> admin only; reports.create -> reporterId == auth.uid
users.read      -> self or admin (PII)
```

2. Issues Found

#	Severity	Issue (before the audit)
1	HIGH	Profile PII readable by any signed-in user: rules allowed <code>users read: if signedIn()</code> — anyone could download the profile
2	MED	Listing <code>createdAt</code> was mutable by the owner on update — a malicious client could backdate or pin its listing to the top
3	MED	Owner status edits were only checked as 'is a string' — arbitrary status values could be written, corrupting feed filters and moderation status
4	MED	The <code>keywords</code> array was unbounded on write — a crafted request could store tens of thousands of entries to interfere with search
5	MED	<code>image0</code> (base64 thumbnail) had no size cap — up to the 1 MB document limit could be written per edit; no per-image size limit
6	MED	Image documents used random ids with no count enforcement — a rule-compliant client could store hundreds of 900 KB image docs under a listing
7	LOW	Self-created profile documents accepted arbitrary extra keys and unbounded string lengths (e.g. planting <code>isAdmin: true</code>)
8	LOW	Reports accepted unbounded free-form fields: unlimited description, arbitrary <code>reason</code> strings, extra keys, literals
9	LOW	Report dialog had no client-side length cap on the description field.
10	INFO	Verified already-secure: no <code>allow ... if true</code> rules; owner edit/delete protection and ownerId immutability correct

3. Issues Fixed

All fixes below were deployed and verified. Client changes are minimal and keep the app simple.

Firestore Rules (server-enforced — deployed to production 2026-10-05)

- **#1:** users read restricted to `isAdmin() || request.auth.uid == uid` (no bulk PII harvesting; the app only ever reads its own profile, so no behaviour change).
- **#2:** listing updates now require `createdAt == resource.data.createdAt`; creates require `createdAt == updatedAt == request.time` (server timestamps only).
- **#3:** owner updates restricted to the canonical status enum `[available, given]` (Arabic literals) (admins bypass for moderation).
- **#4:** keywords must be a list with `<= 3000` entries (worst-case legitimate client output measured $\approx$ 1000; unit-tested).
- **#5:** `image0` capped at 150 000 chars (production max observed 71 440; normal thumbs 15–35 KB).
- **#6:** image document id must equal its index (`string(index) == imgId`, ids '0'..'4') -> hard cap of 5 image documents per listing; the client now writes `images/doc( '$i' )` instead of auto-ids.
- **#7:** profile self-create restricted to the exact register field set (`keys().hasOnly([...])`) with length bounds (name 1–100, etc.); self-update may never touch `disabled` (already denied by key whitelist, now also value-frozen).
- **#8:** reports: fixed key set (`hasOnly`), reason constrained to the 5-value app enum, description `<= 1000` chars (or null), `listingTitle <= 80`, `reporterId == requester`, `createdAt == request.time`.

Client (behaviour-preserving)

- Image writes use deterministic ids '0'–'4' (required by the new count cap).
- Report description input limited to 1000 chars (`LengthLimitingTextInputFormatter`, no visual change).
- Form limits re-verified: title `<= 80`, description `<= 800`, phone regex, `<= 5` images — already matching the rules.

4. Firestore Rules Changes (summary)

Match	Before	After
users/{uid} read	<code>signedIn()</code>	self or admin
users create	<code>disabled == false</code> only	self + exact key set + length bounds
users update (owner)	4 allowed keys	+ name bounds + disabled value-frozen (admin path unchanged)
listings create	owner + validListing + status	+ <code>createdAt/updatedAt == request.time</code> (server stamps)
listings update (owner)	owner + ownerId unchanged + validListing	+ <code>createdAt</code> frozen + status enum + keywords/image0/ownerName caps
listings/images	index 0–4, data $\leq$ 900 000	+ doc id == index -> max 5 docs/listing
reports create	<code>reporterId == self</code> , 2 type checks	+ fixed key set + reason enum + bounded strings + server timestamp
admins/*	write: if false	unchanged (verified secure)

Rules released with `firebase deploy --only firestore:rules` on 2026-10-05 (project ta3awen-mvp-5a5ea). Previous rules backed up before deployment.

5. Image / Data-Size Security

- **Count:** 5 full images max (client picker) and now 5 documents max (rules, id = index).
- **Per-document size:** image payload < 900 000 base64 chars (rules); thumbnail < 150 000; document limit 1 MB remains the platform backstop.
- **Compression:** pick-time resize to 1000px / quality 70; thumbnail 480px / quality 55 — measured production parent documents: max 74 KB, average ≈ 16 KB thumbnail.
- **Quota:** worst-case listing ≈ 4.6 MB of writes (5 x 900 KB + parent) — within free-tier write quota for MVP volumes; no Firebase Storage / billing introduced (constraint respected).
- **Invalid input:** non-string payloads, out-of-range indexes, id/index mismatch, oversized strings and missing required fields are all rejected server-side regardless of client.

6. Authentication / Authorization Findings

- **Identity:** email/password only; anonymous sign-in disabled (verified: anonymous signUp returns ADMIN_ONLY_OPERATION). The old README probe account (probe@test.dz) is dead — no valid credentials exist for it.
- **Admin cannot be self-granted:** admins is write-locked in rules (console/Admin SDK only); profile planting of admin fields is rejected by the key whitelist. Verified by tests.
- **Disabled accounts:** can still complete a login but every Firestore write is denied (canWrite()); verified for listing creation.
- **Client UI hiding is not the control:** admin screens, moderation buttons and report management were re-verified against rules — non-admins are denied at the API even with a modified client.
- **Token handling:** no long-lived credentials in the app; Firebase SDK handles refresh; release build is obfuscated with split debug symbols kept out of git.
- No custom secrets are used by the client — the only key shipped is the Firebase web API key, which is a public identifier by design.

7. Public Repository / Secrets Findings

- `github.com/MoIslam-Dev/ta3awen` (source) is **PRIVATE**; `MoIslam-Dev/ta3awen-apk` is **PUBLIC** and contains only `app-arm64-v8a-release.apk` and PDFs — no code, no config, no credentials (verified file list).
- Git-history scan for signing material: **no** `android/key.properties`, `*.jks/*.keystore`, service-account JSON or `.env` files were ever committed; all are gitignored and only exist on disk.
- `google-services.json` and `lib/firebase_options.dart` are committed **intentionally**: they contain only public identifiers (project id, API key) that ship inside the APK anyway — they are not private credentials. They must not be confused with service-account keys (none exist in the repo).
- README contains a historical test-account password for a deleted probe user — acceptable while the repo stays private; no action required for MVP (noted).
- Obfuscation symbol files (`/symbols/`) and the release keystore stay local-only.

8. Tests Performed

Rules tests — Firestore Emulator, @firebase/rules-unit-testing (50/50 passed)

Security rules file executed against a real emulator with three identities (user A = alice, user B = bob, admin = admin1) plus an unauthenticated context. Scenario matrix, including every authorization case

requested:

Requested scenario	Result
A creates listing (app-shaped write)	ALLOW
B edits A's listing	DENY
B deletes A's listing	DENY
B changes A's listing ownerId	DENY (ownership + ownerId freeze)
A changes own ownerId	DENY
B accesses admin functionality (reports, admin membership, disabling users)	DENY
B modifies / deletes / reads a report	DENY (admin-only)
Self-grant admin (admins doc or planted profile field)	DENY
Report with forged reporterId / junk reason / oversized description	DENY
createdAt tampering (update, literal create, future-dated create)	DENY
Status outside enum (available/given); keywords > 3000; image0 >= 150 000; title > 80	DENY
Image doc id/index mismatch, 6th index, 900 001-char payload	DENY
Cross-user image write; unauthenticated read/write of everything	DENY
Admin: edit/delete others' listings, edit/delete reports, disable users	ALLOW
Registration shape; disabled account write; signed-in image/listing reads	ALLOW

Other verification

- `flutter analyze` — no issues found.
- `flutter test` — 21/21 passed (includes new keyword-cap unit tests proving adversarial max-length texts stay under the 3000-entry rules cap, and the listing-card layout regression test).
- Production data probe (read-only): 107 listings, uniform status, thumbnails <= 71 440 chars, keywords <= 105 entries — all within the new caps, so existing documents remain editable.
- Release build produced and published: `app-arm64-v8a-release.apk`, md5 `94bee51d284d93a3fe0f2082611d9fb3`.

9. Remaining Risks (accepted for MVP)

Risk	Notes / suggested next step
No rate limiting	Rules cannot rate-limit: a client can still spam valid listings or reports (bounded individually). Post-MVP: Cloud Functions.
App Check not enabled	Any holder of the public API key can call the REST APIs directly (still fully rules-bound). Enable Play Integrity / App Check.
Base64 image amplification	Up to ≈ 4.6 MB written per listing; free-tier daily write quota could be exhausted by abuse at scale. Storage migration.
Disabled users can log in	They are fully write-blocked in Firestore; account deletion / auth blocking would need console or Cloud Functions.
imageCount vs actual image docs	Rules cannot count subcollection documents; mitigated by the hard 5-document cap (id = index).
Phone format only length-checked in rules	Regex validation is client-side; forged phone strings remain cosmetically possible.
Seed data	≈100 seed listings still in the feed — delete before public launch (documented cleanup script).
No backups / export	Firebase free tier has no automated backups yet — known MVP priority.
Google account 2FA	Admin membership is controlled by the Firebase/Google console — secure that single account (recommend TOTP/2FA).

10. MVP Security Readiness

Ready for a closed MVP test. The ten checklist requirements are satisfied: cross-user edit/delete/ownerId tampering, privilege escalation, admin-data access and report tampering are all denied server-side (50/50 emulator tests); no permissive rules exist; oversized or malformed documents are rejected; secrets are absent from repositories; and nothing relies on UI hiding alone. Proceed with the seed cleanup, enable App Check when possible, and plan rate limiting + backups before a public launch.

Artifacts: firestore.rules (deployed), rules-test emulator suite (50 tests), commit 42845f2, APK md5 94bee51d284d93a3fe0f2082611d9fb3.